

Agent Wars

AI-Powered Healthcare Monitoring Agent using RAG and Multi-Tool Agentic Workflow

Students

Avishkar More, EGV Santhosh Kumar, Vedant Murarka, Swetha D

Batch

2024 - 2028

Duration

2 Months

Technology

LangChain, FastAPI, FAISS, Supabase, and GCP Cloud Run



PROJECT OVERVIEW

Mission Statement

MedAssist AI is an agentic AI healthcare assistant — **not a simple chatbot** — that reasons over natural language health queries, retrieves reliable medical knowledge via RAG, and dynamically calls tools to deliver personalized, context-aware health insights.

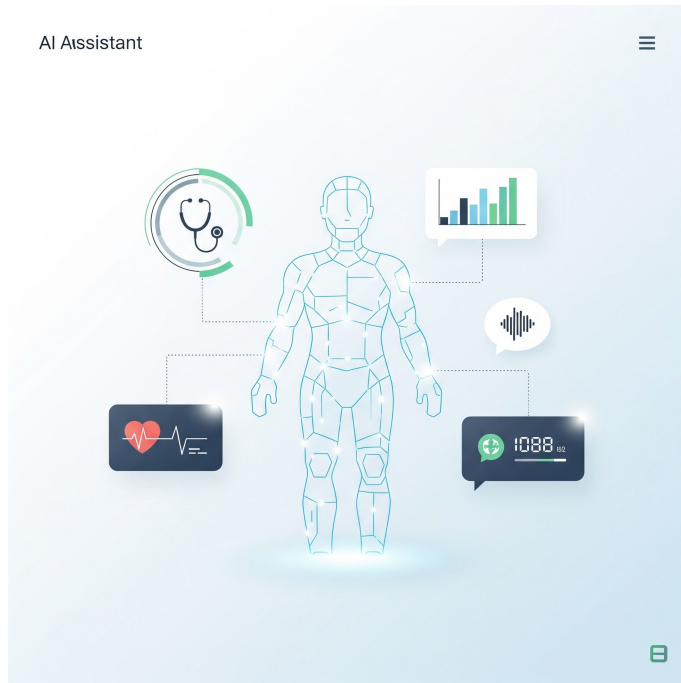
Core Focus Areas

- Medication Adherence
- Drug Interactions
- Health Risk Prediction
- Reliable Medical RAG

How It Works

The LLM Agent Controller analyzes intent, then routes to RAG retrieval, a tool call, or the health database — before generating a grounded, memory-updated response.

Main LangChain Module: AgentExecutor



THE CHALLENGE

Identifying the Friction

Healthcare monitoring is fragmented, reactive, and dependent on manual tracking. Patients struggle with medication adherence, understanding drug interactions, and catching health risks early.

Standard chatbots generate plausible-sounding but unverified answers, risking medical misinformation. Static, rule-based scripts cannot reason, retrieve grounded evidence, or route dynamically.

This drove the need for an agentic system with RAG, dynamic tool-calling, and persistent memory.

01. No Contextual Reasoning

Existing tools cannot distinguish between generic informational queries and personalized clinical tasks like checking drug-to-drug interactions to route intelligently.

02. No Grounding

Without clinical retrieval mechanisms, standard LLMs confidently fabricate responses from training memory, offering no way to verify safety or cite authoritative sources.

03. No Persistent Context

Simple reminder apps record schedules, but lack any medical history continuity, preventing them from synthesizing past insights to deliver personalized guidance.

TECHNOLOGY STACK

Capabl.™



Framework: [LangChain & FastAPI](#)

Orchestration and API integration layer



NLP : [Hugging Face Models](#)

Hugging Face Sentence Transformers · Cross-Encoder Reranker



AI / ML: [Core Libraries](#)

Sentence Transformers · Cross-Encoder · Scikit-learn · NumPy · Pandas



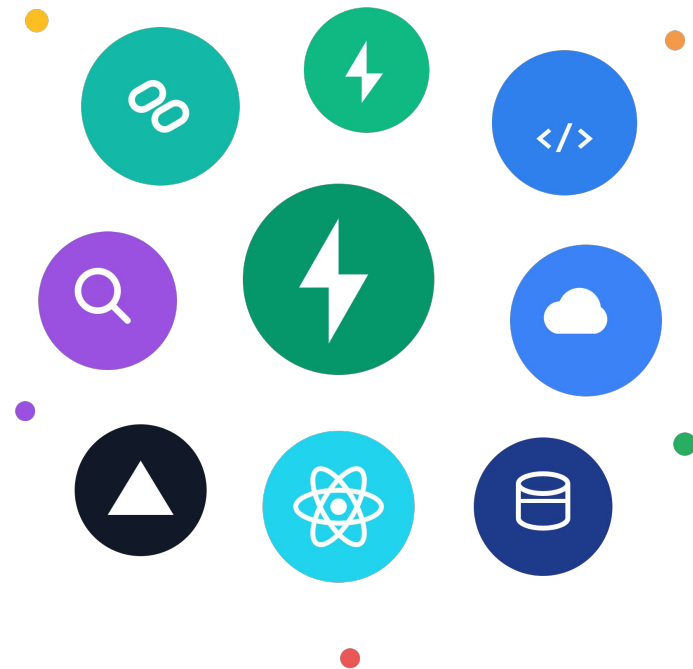
Vector DB: [FAISS + BM25](#)

Hybrid retrieval for fast, contextually grounded search



Hosting: [GCP & Vercel](#)

Backend: Dockerized on GCP Run | Frontend: Vercel Platform



TIMELINE

Architecture & Setup

Define tools, RAG plan, github setup

RAG Pipeline

Build FAISS vector DB & test accuracy

Tool Development

Medication, interaction, risk predictor

Agent Integration

Integrate tools, memory, prototype

Refinement & Eval

Multi-agent refinement, analytics, metrics

Backend & Dashboard

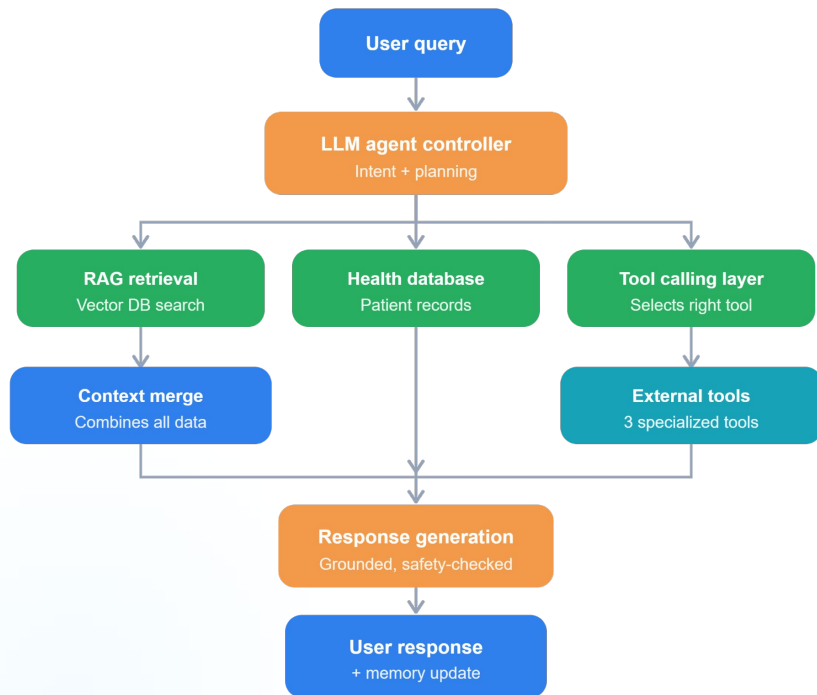
FastAPI backend, React dashboard

Deployment & Demo

Final integration, demo prep & docs



SYSTEM ARCHITECTURE



Technical Workflow

STEP 1: INTERFACE LAYER

Gradio UI (Track A) / React dashboard (Track B) receives the query

STEP 2: AGENT LAYER

LangChain Agent + LLM determines intent and plans the response

STEP 3: RAG & TOOL LAYER

FAISS retrieval and/or Tool Layer (medication reminder, interaction checker, risk predictor) are invoked

STEP 4: DATA LAYER

Supabase PostgreSQL handles authentication and securely persists patient chat history

The LLM Agent Controller handles intent + planning, then branches to RAG Knowledge Retrieval, the Health Database, or the Tool Calling Layer — aggregating context before final response generation.

NOVEL APPROACH

MedAssist AI's X-Factor:



Hybrid RAG Retrieval

FAISS + BM25 + cross-encoder reranking instead of single vector search, for verified, high-accuracy grounding.



Deterministic Tool Calling

Medical tool execution is routed through deterministic Python logic — not LLM generation — preventing hallucinations in safety-critical outputs.



Docker Build Optimization

Engineered a CPU-only PyTorch container, shrinking image size by over 2.5 GB and eliminating heavy CUDA dependencies.



Zero Cold-Start Latency

Hugging Face embedding models are pre-downloaded directly into the Docker image layer during build, preventing Cloud Run timeouts.



Safety Guard Layer

Suppresses ungrounded or irrelevant answers instead of hallucinating, maintaining strict medical guardrails.



Persistent Memory

Supabase-secured authentication combined with chat history to ensure secure continuity across multiple user sessions.

DEMO VIDEO



ROADMAP & FUTURE SCOPE

PHASE 01

Cloud Deployment Upgrade

- Implement Redis for ultra-fast chat history caching
- Add multi-modal capabilities (e.g., uploading PDF lab reports for analysis)
- Migrate from in-memory FAISS to a managed vector DB (Supabase pgvector) for dynamic data updates

PHASE 02

Intelligence & Access Expansion

- Voice-based assistant for hands-free health queries
- Advanced health risk prediction, richer ML models beyond current scikit-learn baseline
- Role-adaptive responses (patient vs. doctor view) with calibrated fallback confidence
- Firebase/Auth0-backed role-based access control (Track B upgrade)

PHASE 03

Scale & Integration

- Real-time IoT integration (wearables, BP/glucose monitors) for continuous health tracking
- Scalable managed vector DB to replace local FAISS for high-concurrency deployment
- CI-integrated API regression testing for reliability at scale
- Industry-level readiness: designed for integration into hospital management systems

Having already deployed to **GCP Cloud Run** with an optimized, cold-start-free container, MedAssist AI's roadmap now focuses on **Redis caching**, multi-modal document analysis, and a dynamic managed vector database.

CONCLUSION



Reliable & Safe

Healthcare Assistant

- 24/7 medical guidance grounded in verified data instead of guesses.
- Built from ~25.8k raw chunks refined to ~23.5k cleaned, indexed chunks.
- Safety guard system actively suppresses ungrounded answers.



Technical Power

Scalable Architecture

- Multi-agent orchestration with hybrid FAISS + BM25 + cross-encoder reranking.
- Full production pipeline: Frontend to FastAPI, Agent System, and Supabase.
- Supports 4 specialized clinical tools and 8 robust REST API endpoints.



Current Status

Known Limitations

- GCP Cloud Run operates on CPU-only. Scaling to large workloads requires GPU instances.
- Heavier model workloads require a transition to a paid tier or GCP/AWS.
- Future roadmap fully outlines migration steps for production deployment.

Built in 2 months on a ₹0, **fully open-source stack**, MedAssist AI proves hybrid RAG and multi-agent routing can deliver a reliable, scalable healthcare assistant.

THANK YOU FOR YOUR ATTENTION • QUESTIONS & FEEDBACK ARE WELCOME